

Parsing

Chapter 3

Outline: How to build a parser?

- Specifying the syntax of a programming language with Context-Free Grammars (CFG)
- Build the parser based on the CFG:
 - Top-Down Parsing
 - Predictive Parsing (LL(k) Parsing)
 - **Bottom-Up Parsing**
 - **LR Parsing**
- More about parsing:
 - Automatic parser generation
 - Error Recovery

Review LL(k) Parsing

- LL(k) parsing is efficient and easy to write by hand
- The weakness of LL(k) parsing is that they must *predict* which production to use, having seen *only the first k tokens* of the right-hand side.

	(	)	\$
S	$S \rightarrow (S)S$	$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

Stack top is T ,
next token is int

Cannot be handled by LL(1) parsing

Bottom-Up Parsing

- **Bottom-Up Parsing:** corresponds to the construction of a parse tree for an input string **beginning at the leaves** (the bottom) and **working up towards the root** (the top)
- **Shift-Reduce Parsing:** a general style of bottom-up parsing

Bottom-Up Parsing

- LR(k) parsing
 - LR grammars are the largest class of grammars for which shift-reduce parsers can be built.
 - More powerful than LL(k) parsing: is able to postpone the decision until it has seen **input tokens corresponding to the entire right-hand side** of the production in question.
 - **LR(k)** Left-to-right parse
 Rightmost derivation
 k -token lookahead
- LALR variant:
 - the basis for parsers for most modern programming languages
 - implemented in tools such as Yacc

Top-Down Parsing

grammar

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

$\text{int} * \text{int} + \text{int}$

E

$T + E$

$T + T$

$T + \text{int}$

$\text{int} * T + \text{int}$

$\text{int} * \text{int} + \text{int}$

productions

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int}$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

- Top-down parsing

Bottom-Up Parsing

grammar

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

int * int + int

int * T + int

T + int

T + T

T + E

E

productions

$T \rightarrow \text{int}$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

$E \rightarrow T$

$E \rightarrow T + E$



- **Idea:** **reduce** the string to the start symbol
- **Reduce** v.s. **Derive**:
 - **Reduce**: replace the RHS of a production with its LHS
- It is important to know **where to reduce**.
 - Cannot reduce the first *int* to *T*. Why?
- Read the input **from left to right** (LR(k))
- **Bottom-up parsing** reduces a string to the start symbol by **inverting** the rightmost derivation. (LR(k))

Bottom-Up Parsing

grammar

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

int * int + int

int * T + int

T + int

T + T

T + E

E

productions

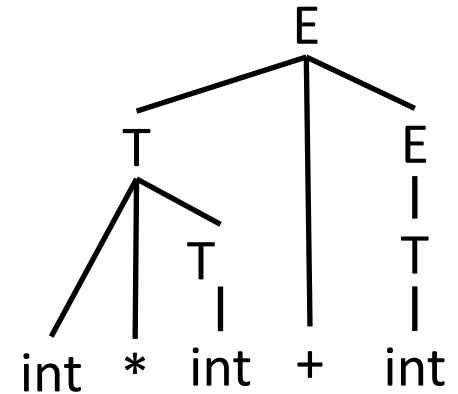
$T \rightarrow \text{int}$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

$E \rightarrow T$

$E \rightarrow T + E$



- **Idea:** **reduce** the string to the start symbol
- **Reduce** v.s. **Derive**:
 - **Reduce**: replace the RHS of a production with its LHS
- **Bottom-up parsing**: from the bottom of the parse tree to the root.
 - if we can reduce to the start symbol, then we successfully parse the string.

LR Parsing

α . β : **left** substring has **terminals** and **non-terminals**, right substring contains only **terminals**.

$S \rightarrow E \$$

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

. int * int + int \$
int * T . + int \$
T . + int \$
T + T . \$
T + E . \$
E . \$

LR Parsing

$\alpha . \beta$: left substring has terminals and non-terminals, right substring contains only terminals.

$S \rightarrow E \$$
 $E \rightarrow T + E$
 $E \rightarrow T$
 $T \rightarrow \text{int} * T$
 $T \rightarrow \text{int}$

Input

. int * int + int \$	. int * int + int \$
	int
	int *
	int * int
int * T . + int \$	int * T
T . + int \$	T
	T +
	T + int
T + T . \$	T + T
T + E . \$	T + E
E . \$	E

LR Parsing

$\alpha . \beta$: left substring has terminals and non-terminals, right substring contains only terminals.

Stack

“parse state”

Input

$S \rightarrow E \$$

$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow \text{int} * T$

$T \rightarrow \text{int}$

. int * int + int \$
int * T . + int \$
T . + int \$
T + T . \$
T + E . \$
E . \$

	. int * int + int \$
int	. * int + int \$
int *	. int + int \$
int * int	. + int \$
int * T	. + int \$
T	. + int \$
T +	. int \$
T + int	. \$
T + T	. \$
T + E	. \$
E	. \$

LR Parsing

- Parser keeps track of
 - position in current input (what input to read next)
 - a stack of terminal & non-terminal symbols representing the “parse so far”
- The parser performs several actions:
 - **shift**: push next input on to top of stack
 - **reduce R**:
 - top of stack should match RHS of rule R (e.g., $X \rightarrow A B C$)
 - pop the RHS from the top of stack (e.g., pop C B A)
 - push the LHS onto the stack (e.g., push X)
 - **accept**: shift \$ and can reduce what remains on stack to start symbol
 - **error**
- **How does the LR parser know when to shift and when to reduce?**

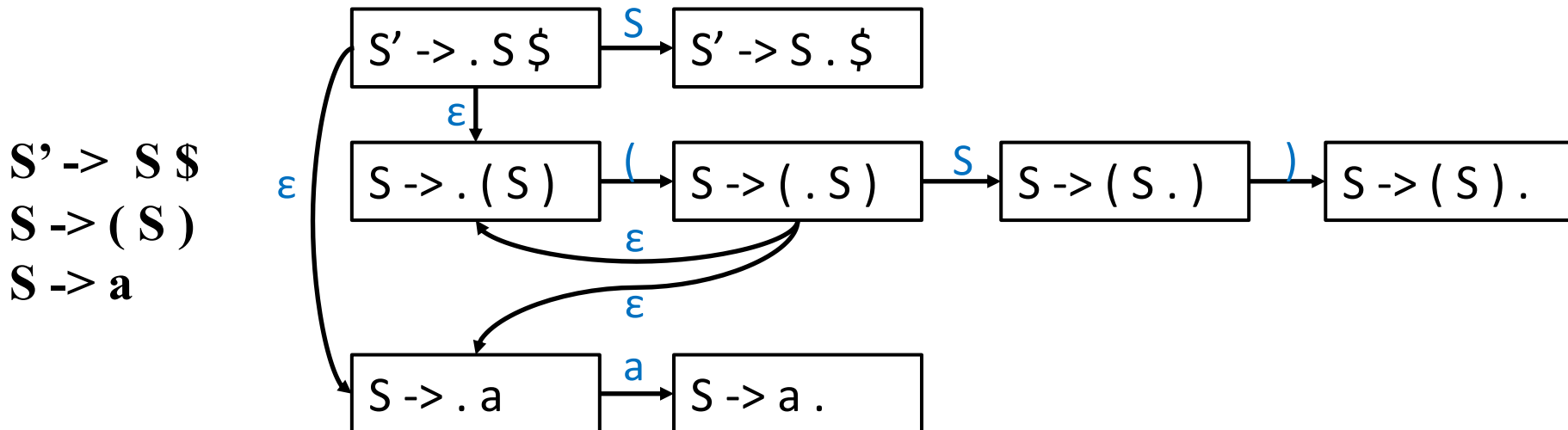
LR(0) Parsing

Start from the simplest LR parsing, i.e., LR(0) Parsing

- LR(0) grammars are those can be parsed looking only at the stack, making shift/reduce decisions without any lookahead.
- How to make such decisions?
- Idea: summarize the “parsing state” and enumerate all valid parsing states and the transactions between them based on the grammar.

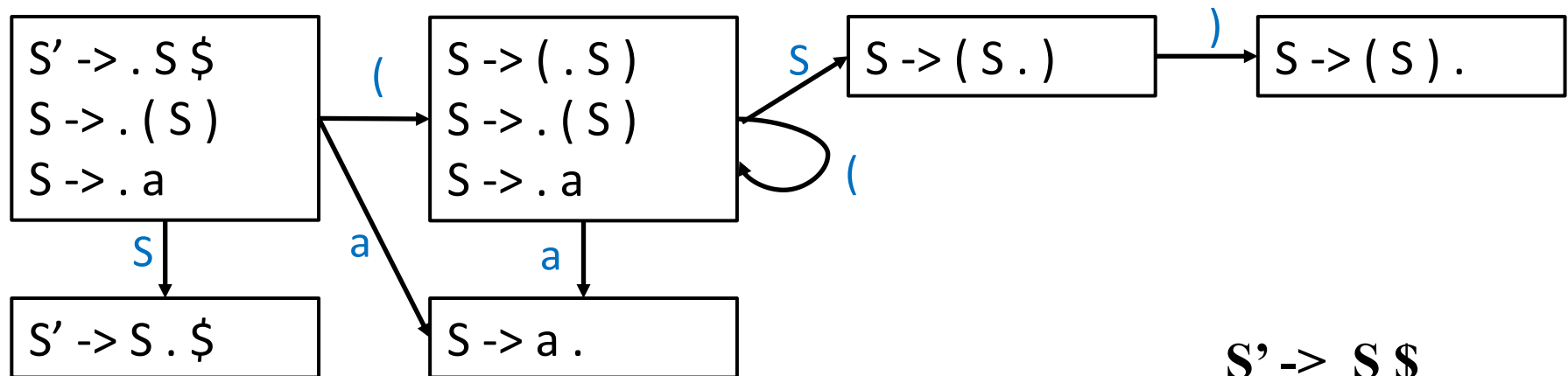
LR(0) Parsing

- $\cdot$: the current position of the parser
- start with $S' \rightarrow \cdot S \$$: stack should be empty and the input is expected to be a complete S-sentence followed by $\$$
- $A \rightarrow \alpha \cdot \beta$: LR(0) item
 - means the parser have processed α and would like to see β next.
- We obtain an NFA
- We can convert it into a DFA through ϵ -closure and state merging



LR(0) Parsing

- The state of the DFA is a set of items
- This NFA is not that complex.
 - Only epsilon edges.
- We can also directly construct this DFA based on a procedure named Closure.



$S' \rightarrow S S$
 $S \rightarrow (S)$
 $S \rightarrow a$

LR(0) Parsing

I: a set of items

X: a symbol (terminal or non-terminal)

T: the set of states

E: the set of (shift or goto) edges

```
Closure( $I$ ) =  
  repeat  
    for any item  $A \rightarrow \alpha.X\beta$  in  $I$   
      for any production  $X \rightarrow \gamma$   
         $I \leftarrow I \cup \{X \rightarrow .\gamma\}$   
  until  $I$  does not change.  
  return  $I$ 
```

```
Goto( $I, X$ ) =  
  set  $J$  to the empty set  
  for any item  $A \rightarrow \alpha.X\beta$  in  $I$   
    add  $A \rightarrow \alpha X.\beta$  to  $J$   
  return Closure( $J$ )
```

Initialize T to $\{\mathbf{Closure}(\{S' \rightarrow .S\})\}$

Initialize E to empty.

```
repeat  
  for each state  $I$  in  $T$   
    for each item  $A \rightarrow \alpha.X\beta$  in  $I$   
      let  $J$  be Goto( $I, X$ )  
       $T \leftarrow T \cup \{J\}$   
       $E \leftarrow E \cup \{I \xrightarrow{X} J\}$   
until  $E$  and  $T$  did not change in this  
iteration
```

1. calculate the new state
2. store the state
3. store the edge

LR(0) Parsing

0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L , S$

Initialize T to $\{\text{Closure}(\{S' \rightarrow .S \$\})\}$

Initialize E to empty.

repeat

for each state I in T

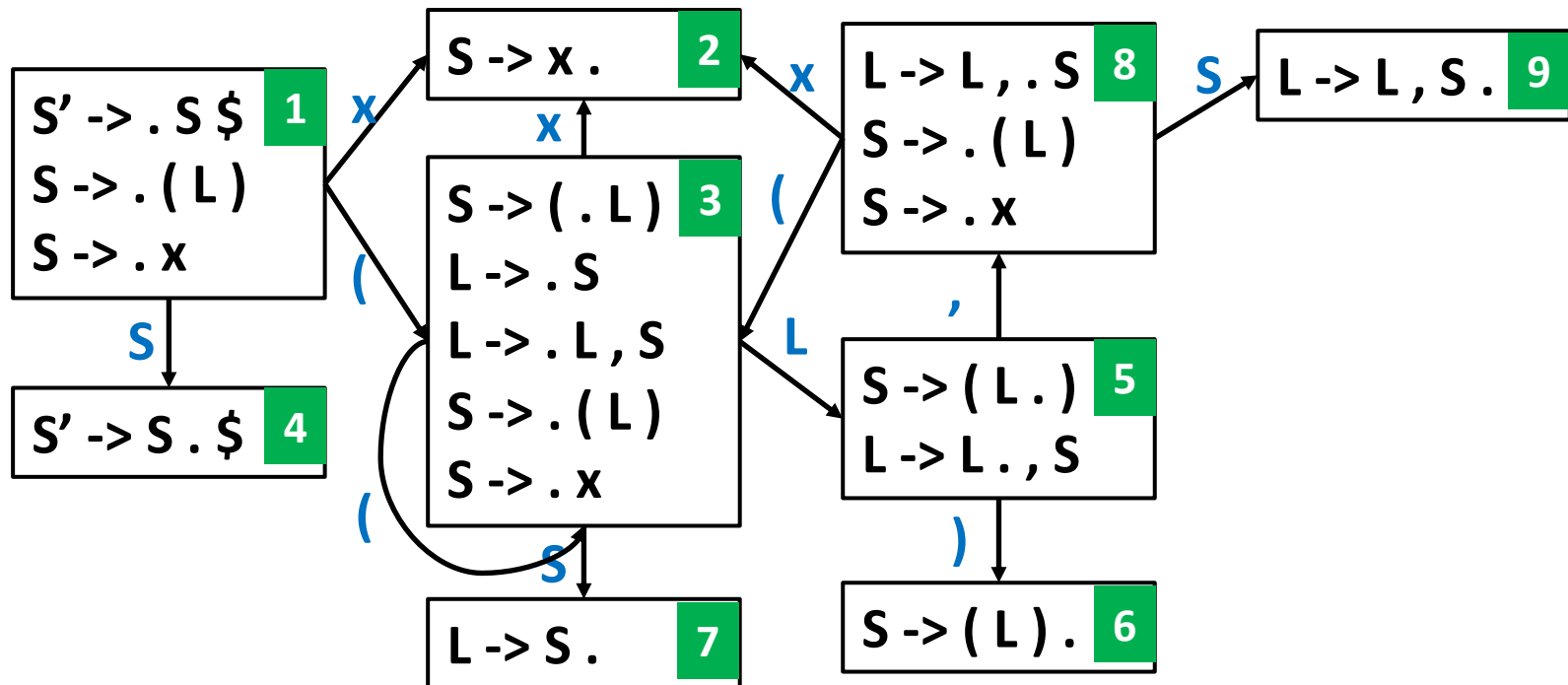
for each item $A \rightarrow \alpha.X\beta$ in I

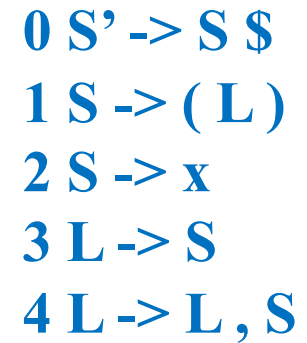
 let J be **Goto**(I, X)

$T \leftarrow T \cup \{J\}$

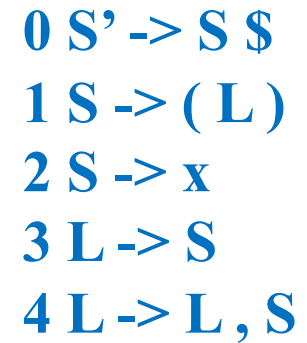
$E \leftarrow E \cup \{I \xrightarrow{X} J\}$

until E and T did not change in this iteration

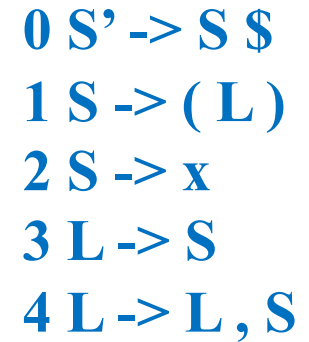




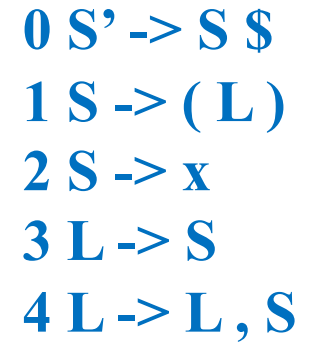
- 0 $S' \rightarrow S \$$
- 1 $S \rightarrow (L)$
- 2 $S \rightarrow x$
- 3 $L \rightarrow S$
- 4 $L \rightarrow L, S$



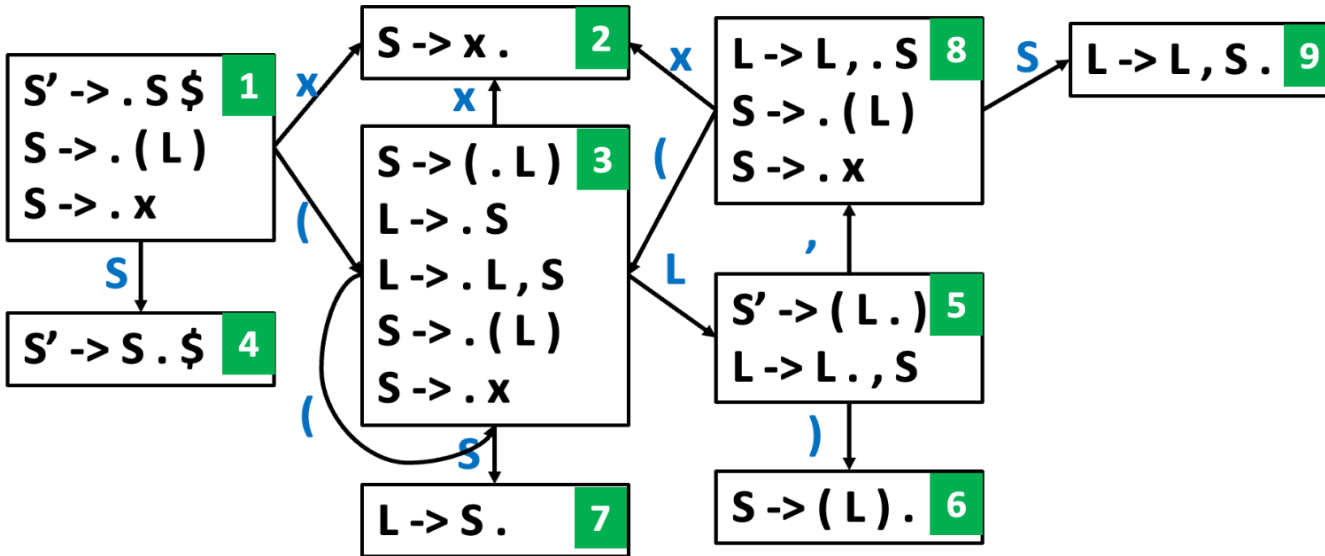
- 0 $S' \rightarrow S \$$
- 1 $S \rightarrow (L)$
- 2 $S \rightarrow x$
- 3 $L \rightarrow S$
- 4 $L \rightarrow L, S$



- 0 $S' \rightarrow S \$$
- 1 $S \rightarrow (L)$
- 2 $S \rightarrow x$
- 3 $L \rightarrow S$
- 4 $L \rightarrow L, S$

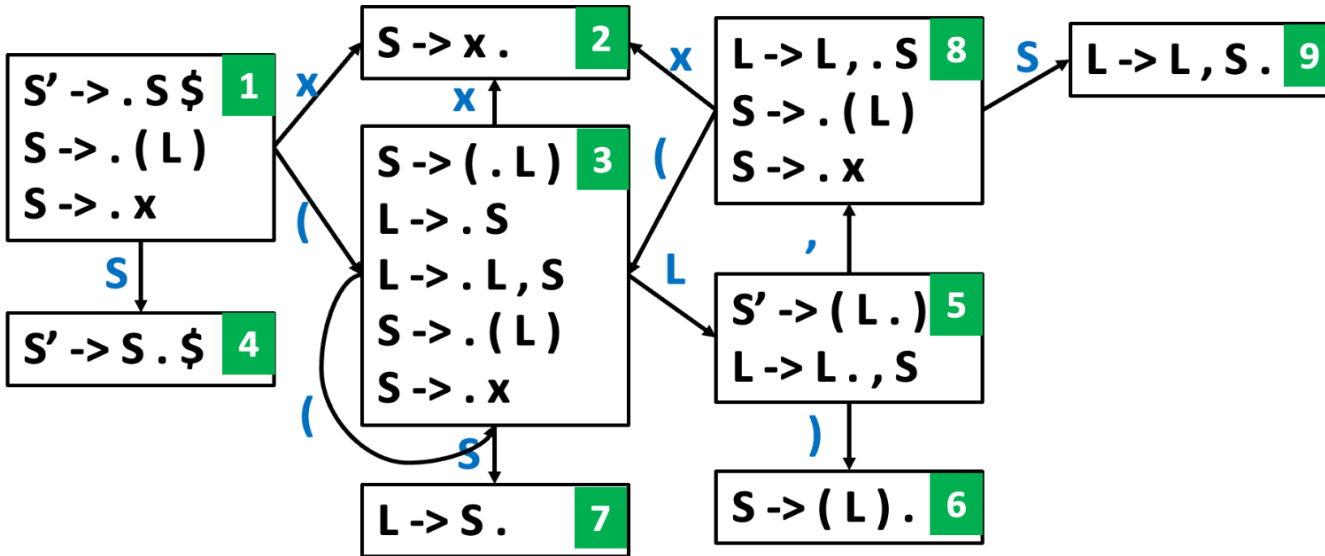


- 0 $S' \rightarrow S \$$
- 1 $S \rightarrow (L)$
- 2 $S \rightarrow x$
- 3 $L \rightarrow S$
- 4 $L \rightarrow L, S$



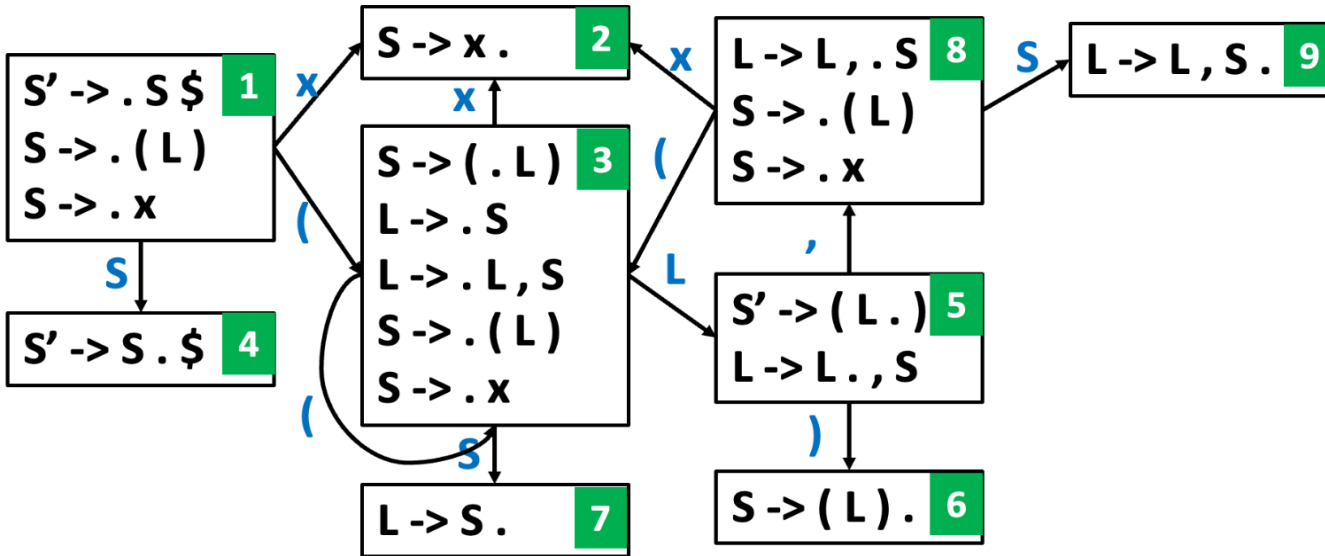
0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L, S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	



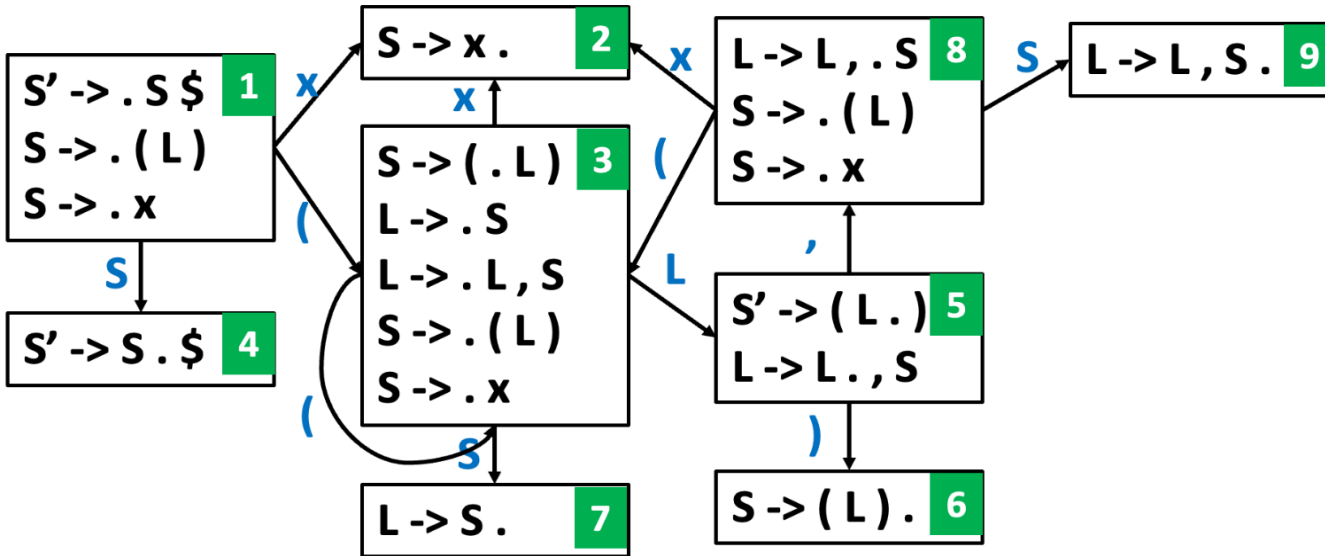
0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L, S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	



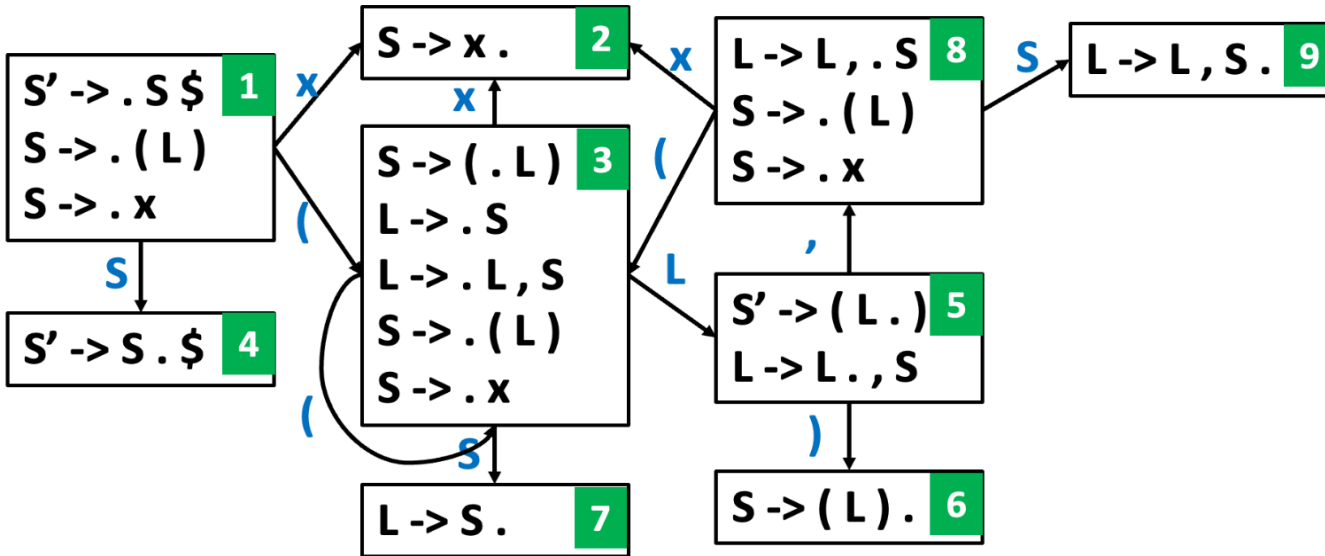
0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L, S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	



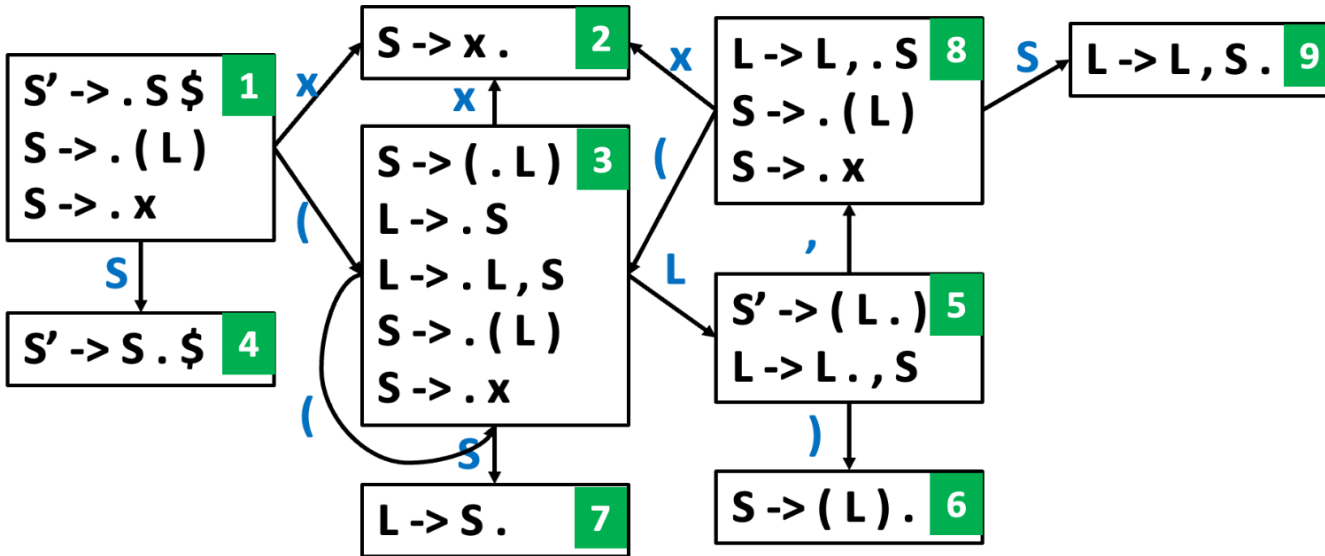
0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L, S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	shift 6
1, 3, 5, 6	(L)	\$	



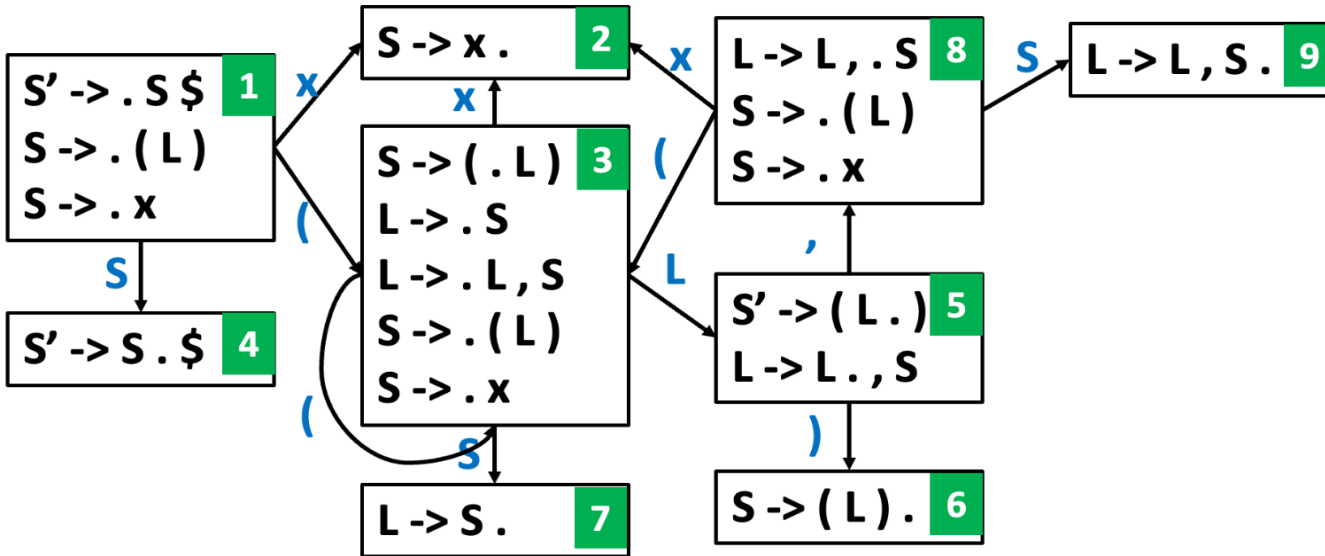
0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L, S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	shift 6
1, 3, 5, 6	(L)	\$	reduce 1 $S \rightarrow (L)$
1	S	\$	



0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L , S$

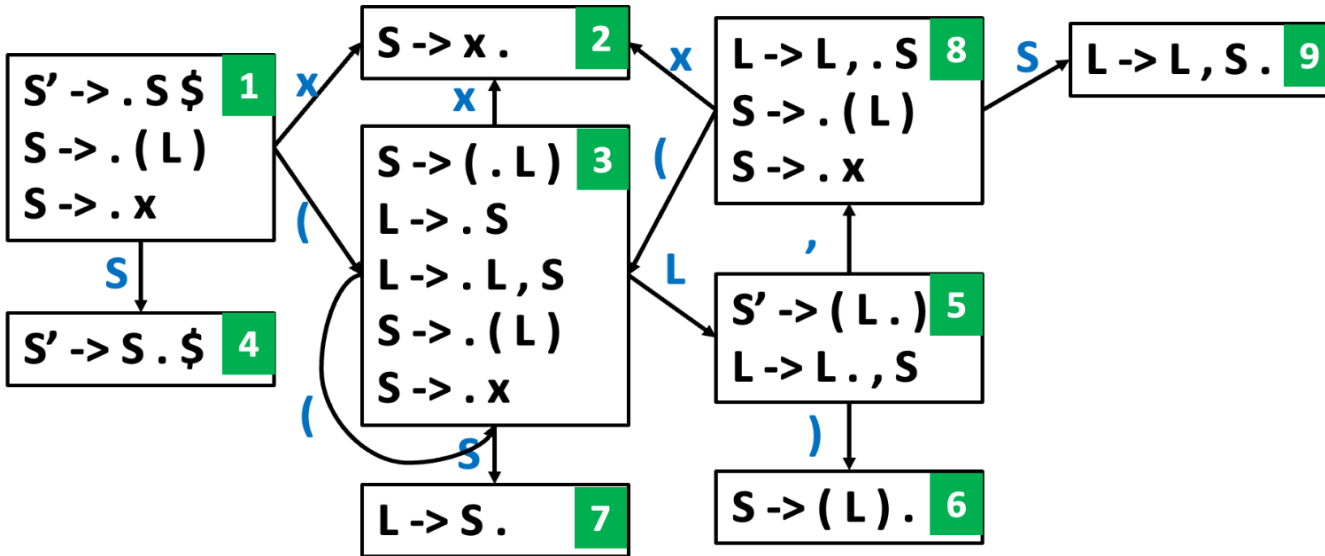
Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	shift 6
1, 3, 5, 6	(L)	\$	reduce 1 $S \rightarrow (L)$
1	S	\$	goto 4
1, 4	S	\$	



0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L , S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	shift 6
1, 3, 5, 6	(L)	\$	reduce 1 $S \rightarrow (L)$
1	S	\$	goto 4
1, 4	S	\$	accept

The two stacks can also be merged into one to store the pairs of states and symbols.

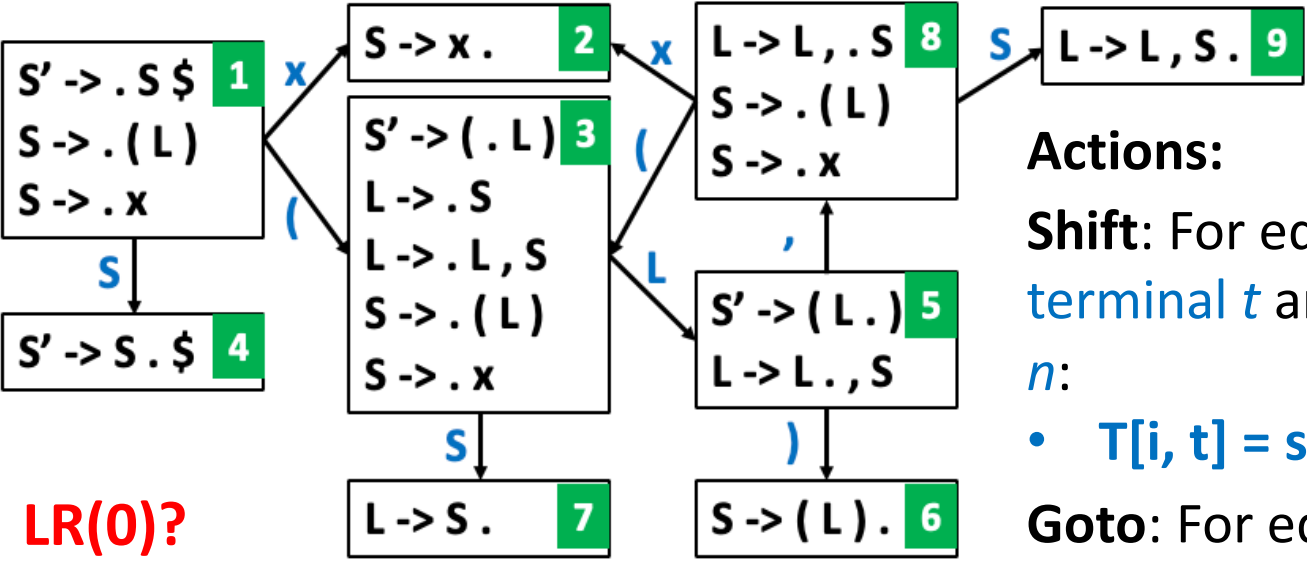


0 $S' \rightarrow S \$$
 1 $S \rightarrow (L)$
 2 $S \rightarrow x$
 3 $L \rightarrow S$
 4 $L \rightarrow L , S$

Stack (states)	Stack (symbols)	Input	Action
1		(x) \$	shift 3
1, 3	(	x) \$	shift 2
1, 3, 2	(x	) \$	reduce 2 $S \rightarrow x$
1, 3	(S	) \$	goto 7
1, 3, 7	(S	) \$	reduce 3 $L \rightarrow S$
1, 3	(L	) \$	goto 5
1, 3, 5	(L	) \$	shift 6
1, 3, 5, 6	(L)	\$	reduce 1 $S \rightarrow (L)$
1	S	\$	goto 4
1, 4	S	\$	accept

Four types of actions:

1. Shift
2. Reduce
3. Goto
4. Accept



LR(0)?

	(	)	x	,	\$	S	L
1	s3		s2			g4	
2	r2	r2	r2	r2	r2		
3	s3		s2			g7	g5
4					a		
5		s6		s8			
6	r1	r1	r1	r1	r1		
7	r3	r3	r3	r3	r3		
8	s3		s2			g9	
9	r4	r4	r4	r4	r4		

Actions:

Shift: For edge labeled with terminal t and from state i to state n :

- $T[i, t] = sn$ (shift n)

Goto: For edge labeled with non-terminal X and from state i to state n :

- $T[i, X] = gn$ (goto n)

Reduce: For item in a state i with dot at the end (e.g., $X \rightarrow A \dots C \cdot$):

- $T[i, \text{each terminal}] = rk$ (reduce k)
- k is the **index of this production**

Accept: For each state i containing $S' \rightarrow S \cdot \$$:

- $T[i, \$] = \text{accept}$

LR(0) Parsing – Parsing Algorithm

Look up **top stack state** and **input symbol** to get action; If action is:

Shift(n): Advance input **one token**;
push **n** on stack.

Reduce(k): Pop stack as many times **as the number of symbols** on the RHS of **rule k**;
Let **X** be the LHS symbol of **rule k**;
In **the state** now on top of stack, look up **X** to get “**goto n**”;
Push **n** on top of stack.

Accept: Stop parsing, report success.

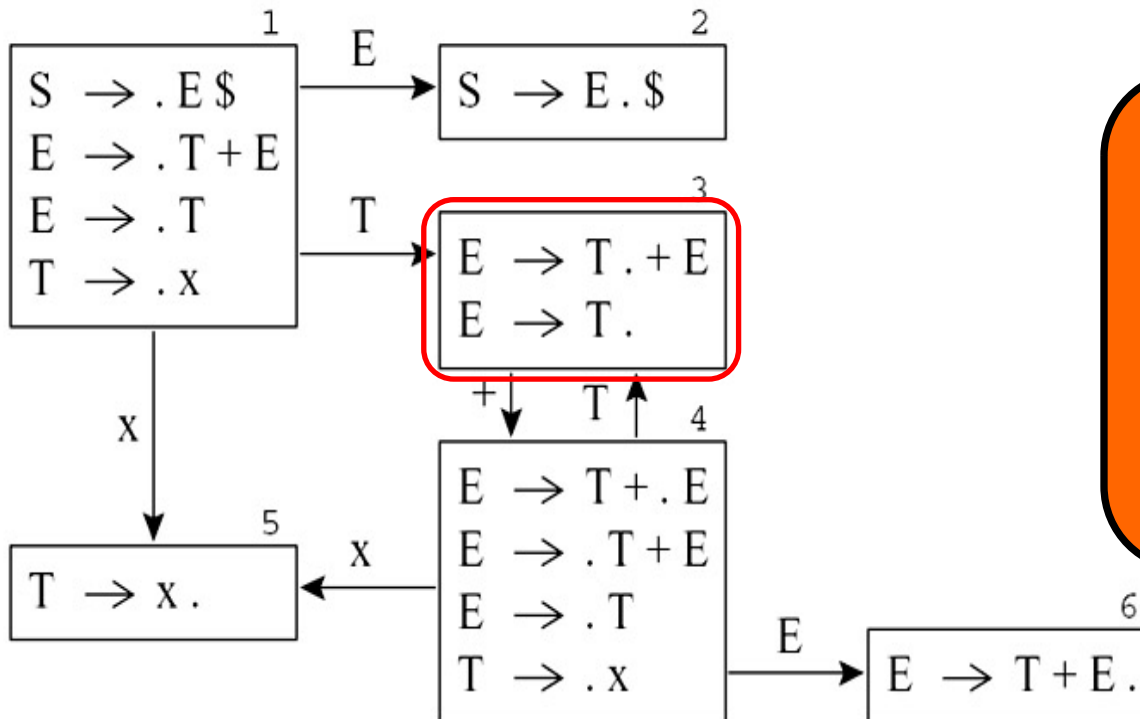
Error: Stop parsing, report failure.

- Only the **state stack** is used
- This is a **general algorithm** and can be used by other LR parsing
- For **LR(0)**, we do not need to look up input symbol to know whether we should **shift or reduce**.

SLR Parsing

$0 S \rightarrow E \$$
 $1 E \rightarrow T + E$
 $2 E \rightarrow T$
 $3 T \rightarrow x$

	x	+	\$	E	T
1	s5			g2	g3
2			a		
3	r2	s4,r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		

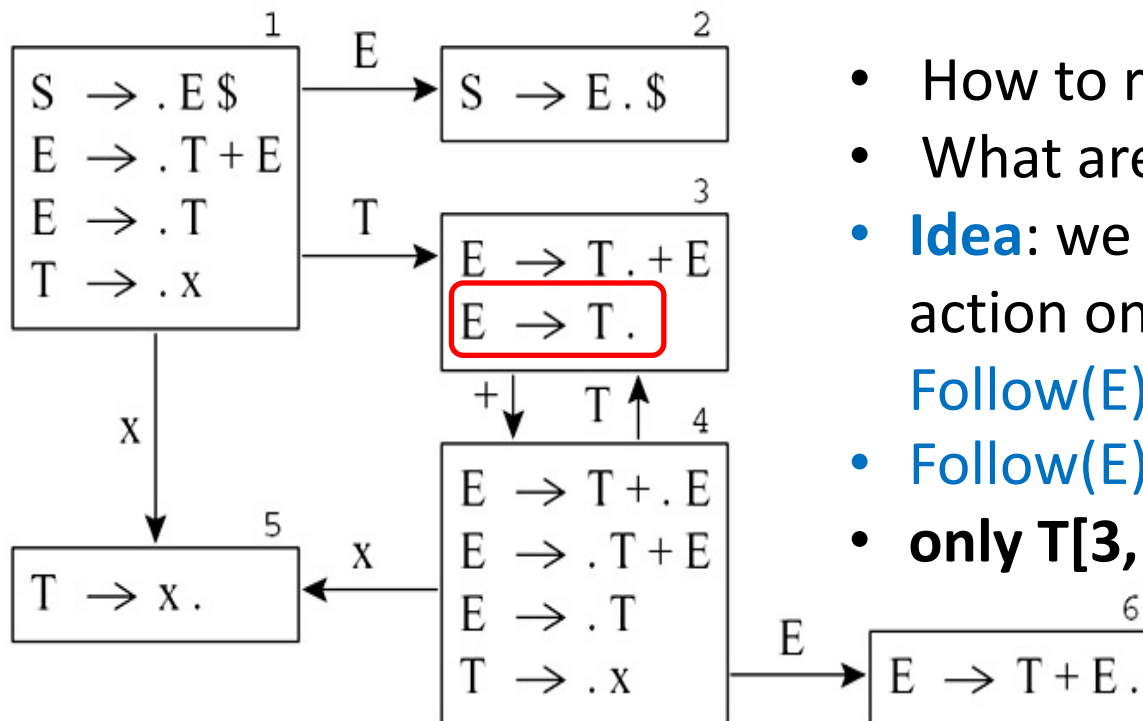


This is a conflict and indicates that the grammar is not LR(0) - it cannot be parsed by an LR(0) parser.

SLR Parsing

$0 S \rightarrow E \$$
 $1 E \rightarrow T + E$
 $2 E \rightarrow T$
 $3 T \rightarrow x$

	x	+	\$	E	T
1	s5			g2	g3
2			a		
3	r2	s4, r2	r2		
4	s5			g6	g3
5	r3	r3	r3		
6	r1	r1	r1		



- How to resolve this conflict?
- What are the valid next tokens for $r2$?
- **Idea:** we can choose the reduce action only if the next input token $t \in \text{Follow}(E)$
- $\text{Follow}(E) = \{\$, \}$
- **only $T[3, \$]$ can be $r2$!**

SLR Parsing

SLR Parsing: Simple LR Parsing

Put reduce actions into the table only where indicated by the FOLLOW set.

```
 $R \leftarrow \{\}$   
for each state  $I$  in  $T$   
  for each item  $A \rightarrow \alpha.$  in  $I$   
    for each token  $X$  in FOLLOW( $A$ )  
       $R \leftarrow R \cup \{(I, X, A \rightarrow \alpha)\}$ 
```

The algorithm for putting reduce actions into an SLR table

I : the row

X : the column

$A \rightarrow \alpha$: the production

	x	+	\$	E	T
1	s5			g2	g3
2			a		
3		s4	r2		
4	s5			g6	g3
5		r3	r3		
6			r1		

LR(1) Parsing

- There can still be conflicts in SLR parsing tables

0 $S' \rightarrow S \$$

1 $S \rightarrow V = E$

2 $S \rightarrow E$

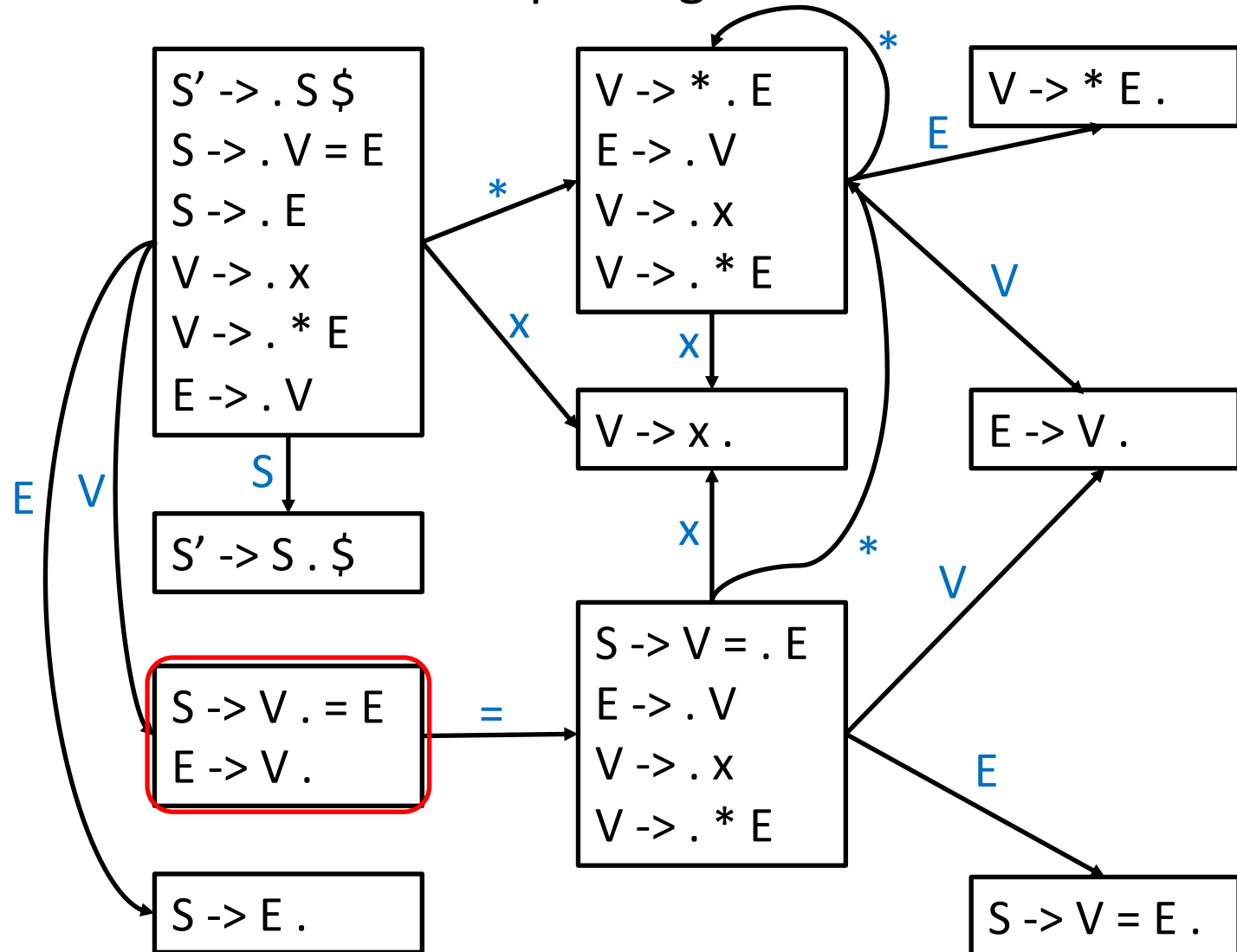
3 $E \rightarrow V$

4 $V \rightarrow x$

5 $V \rightarrow * E$

Follow(E) = { $\$, =$ }

conflict!



LR(1) Parsing

- LR(1) parsing is more powerful than SLR parsing.
 - LR(0): chooses shift or reduce only based on the current state
 - SLR: chooses shift or reduce based on more information (whether the next token is in the specific Follow set)
- The idea of LR(1): add more information into the state of DFA
- How does it resolve these conflicts?

LR(1) Parsing

- There can still be conflicts in SLR parsing tables

0 $S' \rightarrow S \$$

1 $S \rightarrow V = E$

2 $S \rightarrow E$

3 $E \rightarrow V$

4 $V \rightarrow x$

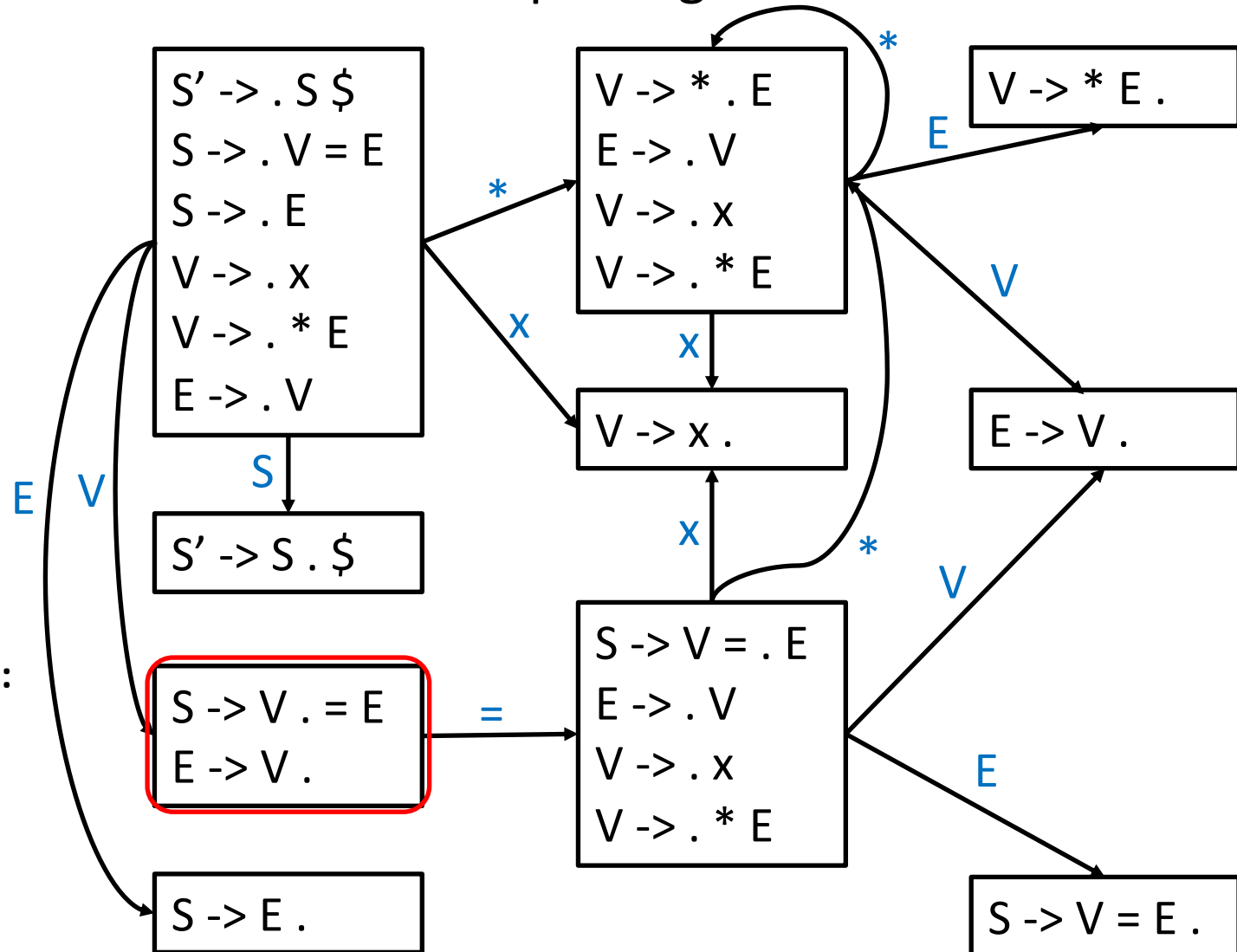
5 $V \rightarrow * E$

Follow(E) = { $\$, =$ }

Current parsing state:

$V . = x$

Shift or Reduce?



LR(1) Parsing

- LR(1) parsing is more powerful than SLR parsing.
- **Idea**: add more information into the state of DFA
 - Specify which terminal symbols can appear after **the whole RHS** of the production in certain context.
 - Distinguish different contexts.
- Also lead to more states.

LR(1) Parsing

- An LR(1) item consists of an LR(0) item and a lookahead symbol.
 - $(A \rightarrow \alpha.\beta, x)$
 - We record the token after **the whole RHS** of this production
- An item $(A \rightarrow \alpha.\beta, x)$ indicates that the sequence α is on top of the (symbol) stack, and at the head of the input is a string derivable from βx .
- For item $(A \rightarrow \alpha.\beta, x)$, the next input token should be in $\text{First}(\beta x)$
 - β can be empty
- Compared to SLR, LR(1) tracks the lookahead symbol.

LR(1) Parsing

```
Closure( $I$ ) =  
  repeat  
    for any item  $(A \rightarrow \alpha.X\beta, z)$  in  $I$   
      for any production  $X \rightarrow \gamma$   
        for any  $w \in \text{FIRST}(\beta z)$   
           $I \leftarrow I \cup \{(X \rightarrow .\gamma, w)\}$   
    until  $I$  does not change  
return  $I$ 
```

LR(1) Closure

1. Change the notion of item
2. When performing ϵ -transactions, record the context on the right (i.e., w)

```
Closure( $I$ ) =  
  repeat  
    for any item  $A \rightarrow \alpha.X\beta$  in  $I$   
      for any production  $X \rightarrow \gamma$   
         $I \leftarrow I \cup \{X \rightarrow .\gamma\}$   
  until  $I$  does not change.  
return  $I$ 
```

LR(0) Closure

The start state is the closure of the item $(S' \rightarrow .S \$, ?)$

LR(1) Parsing

Goto(I, X) =
 $J \leftarrow \{\}$
 for any item $(A \rightarrow \alpha.X\beta, z)$ in I
 add $(A \rightarrow \alpha X.\beta, z)$ to J
 return **Closure**(J).

LR(1) Goto

Only change the notion of item

Goto(I, X) =
 set J to the empty set
 for any item $A \rightarrow \alpha.X\beta$ in
 add $A \rightarrow \alpha X.\beta$ to J
 return **Closure**(J)

LR(0) Goto

LR(1) Parsing

```
 $R \leftarrow \{ \}$   
for each state  $I$  in  $T$   
  for each item  $(A \rightarrow \alpha., z)$  in  $I$   
     $R \leftarrow R \cup \{(I, z, A \rightarrow \alpha)\}$ 
```

LR(1) Reduce Actions

Add z in items and reduce actions

```
 $R \leftarrow \{ \}$   
for each state  $I$  in  $T$   
  for each item  $A \rightarrow \alpha.$  in  $I$   
    for each token  $X$  in  
    FOLLOW( $A$ )  
       $R \leftarrow R \cup \{(I, X, A \rightarrow \alpha)\}$ 
```

SLR Reduce Actions

- The action $(I, z, A \rightarrow \alpha)$ indicates that in state I , on lookahead symbol z , the parser will reduce by rule $A \rightarrow \alpha$.
- Some tokens in FOLLOW(A) may not always be valid in the states with $A \rightarrow \alpha$.
- The lookahead token of LR(1) is more accurate than that of SLR.

LR(1) Parsing

- Example of an LR(1) state

0 $S' \rightarrow S \$$
1 $S \rightarrow V = E$
2 $S \rightarrow E$
3 $E \rightarrow V$
4 $V \rightarrow x$
5 $V \rightarrow * E$

$S' \rightarrow . S \$$?
 $S \rightarrow . V = E$ \$
 $S \rightarrow . E$ \$
 $V \rightarrow . x$ =
 $V \rightarrow . * E$ =
 $E \rightarrow . V$ \$
 $V \rightarrow . x$ \$
 $V \rightarrow . * E$ \$

$S' \rightarrow . S \$$?
 $S \rightarrow . V = E$ \$
 $S \rightarrow . E$ \$
 $V \rightarrow . x$ \$, =
 $V \rightarrow . * E$ \$, =
 $E \rightarrow . V$ \$

LR(1) Parsing

- This grammar is not a SLR grammar, but a LR(1) grammar

$$0 \ S' \rightarrow S \ \$$$

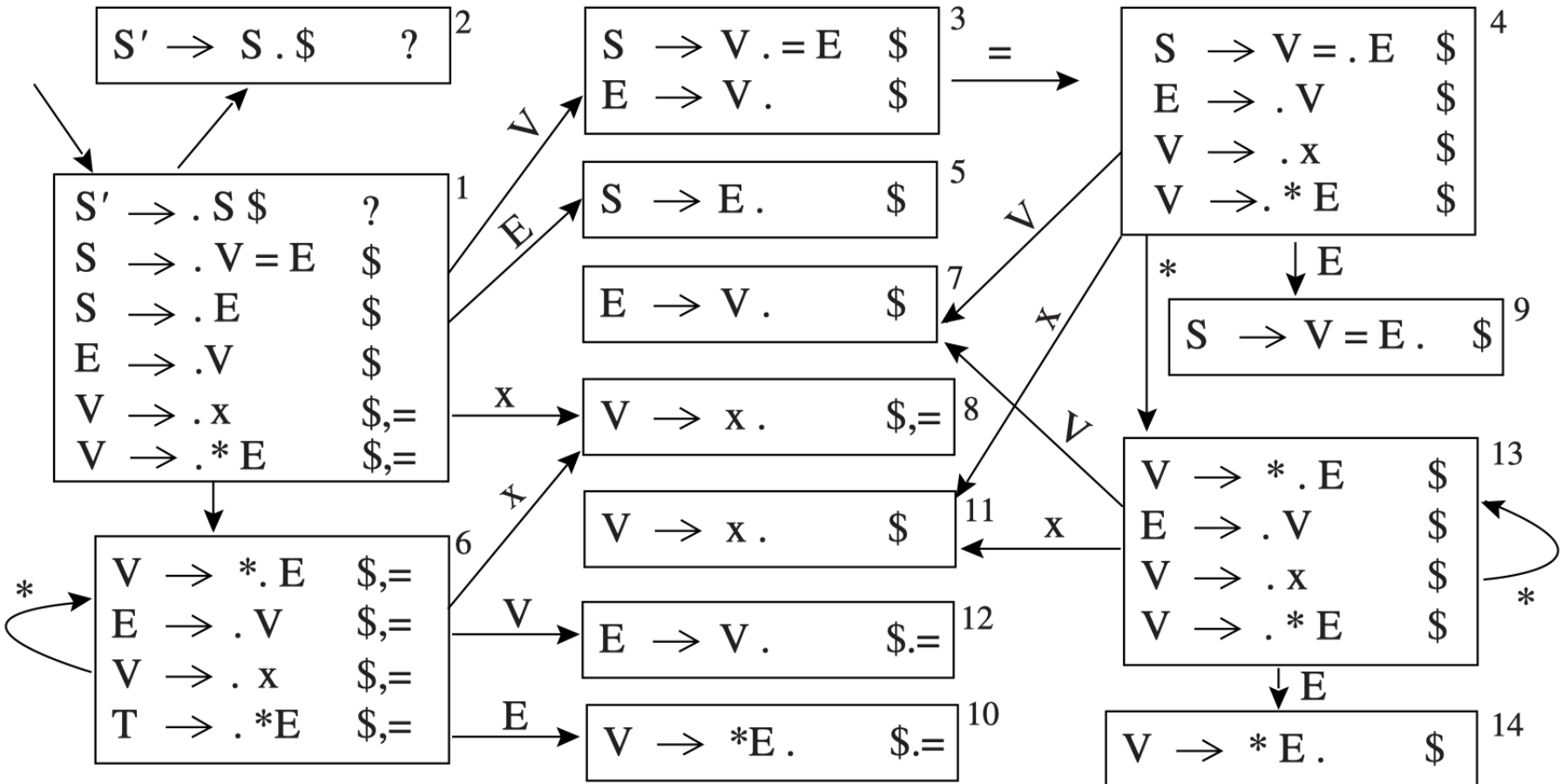
1 $\mathbf{S} \rightarrow \mathbf{V} = \mathbf{E}$

2 S -> E

3 E -> V

4 V -> x

5 V -> * E



LR(1) Parsing

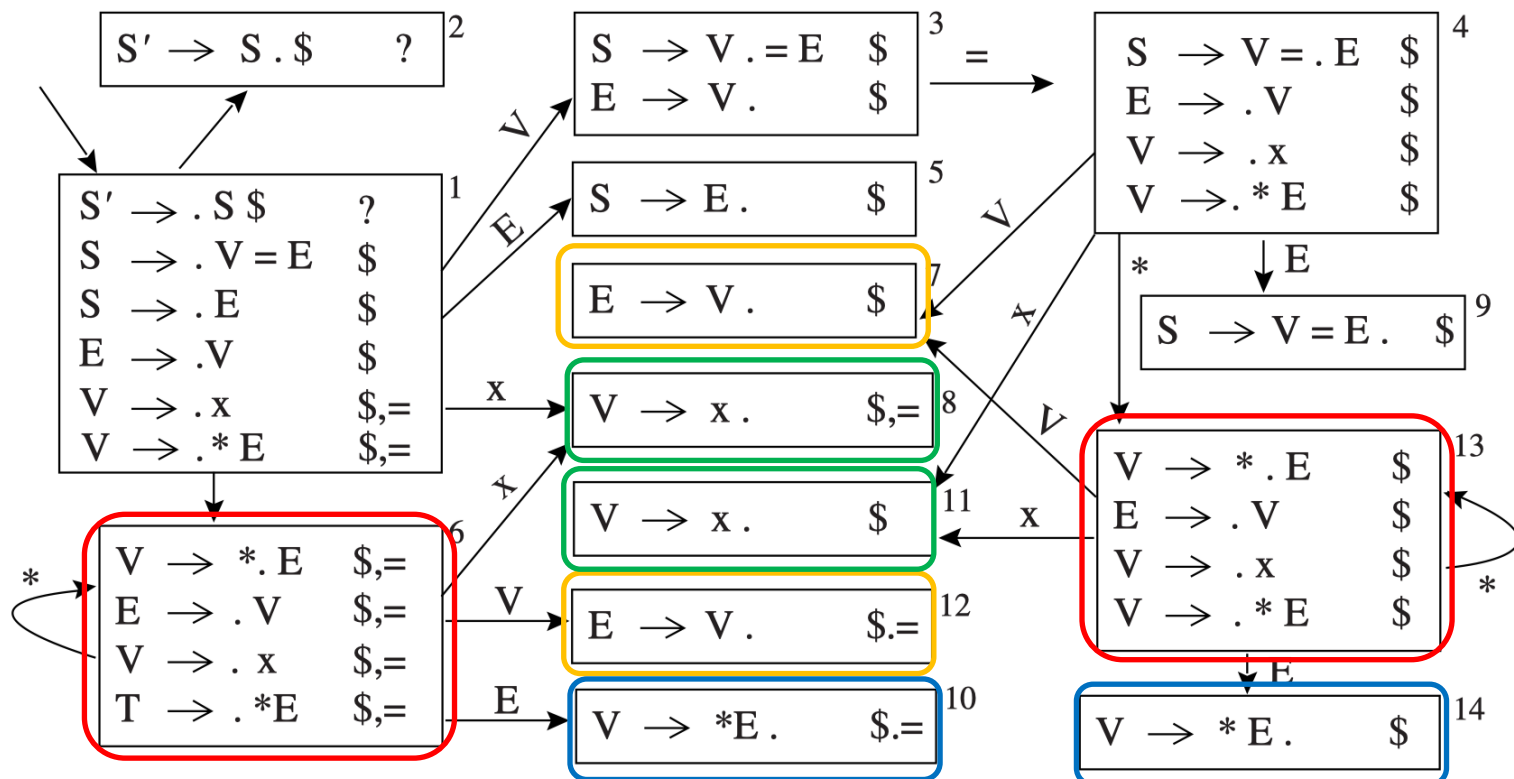
0 $S' \rightarrow S \$$
 1 $S \rightarrow V = E$
 2 $S \rightarrow E$
 3 $E \rightarrow V$
 4 $V \rightarrow x$
 5 $V \rightarrow * E$

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5				r2			
6	s8	s6				g10	g12
7				r3			
8			r4	r4			
9				r1			
10			r5	r5			
11				r4			
12			r3	r3			
13	s11	s13				g14	g7
14				r5			

(a) LR(1)

LALR(1) Parsing

- LR(1) parsing tables can be very large, with many states.
- **LALR(1) parsing**: the parsing table is made by merging any two states **whose items are identical except for lookahead sets** in the LR(1) parsing table.
 - Look-Ahead LR(1)



LALR(1) Parsing

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s11	s13				g9	g7
5				r2			
6	s8	s6				g10	g12
7				r3			
8			r4	r4			
9				r1			
10			r5	r5			
11				r4			
12			r3	r3			
13	s11	s13				g14	g7
14				r5			

(a) LR(1)

	x	*	=	\$	S	E	V
1	s8	s6			g2	g5	g3
2				a			
3			s4	r3			
4	s8	s6				g9	g7
5				r2			
6	s8	s6				g10	g7
7			r3	r3			
8			r4	r4			
9				r1			
10			r5	r5			

(b) LALR(1)

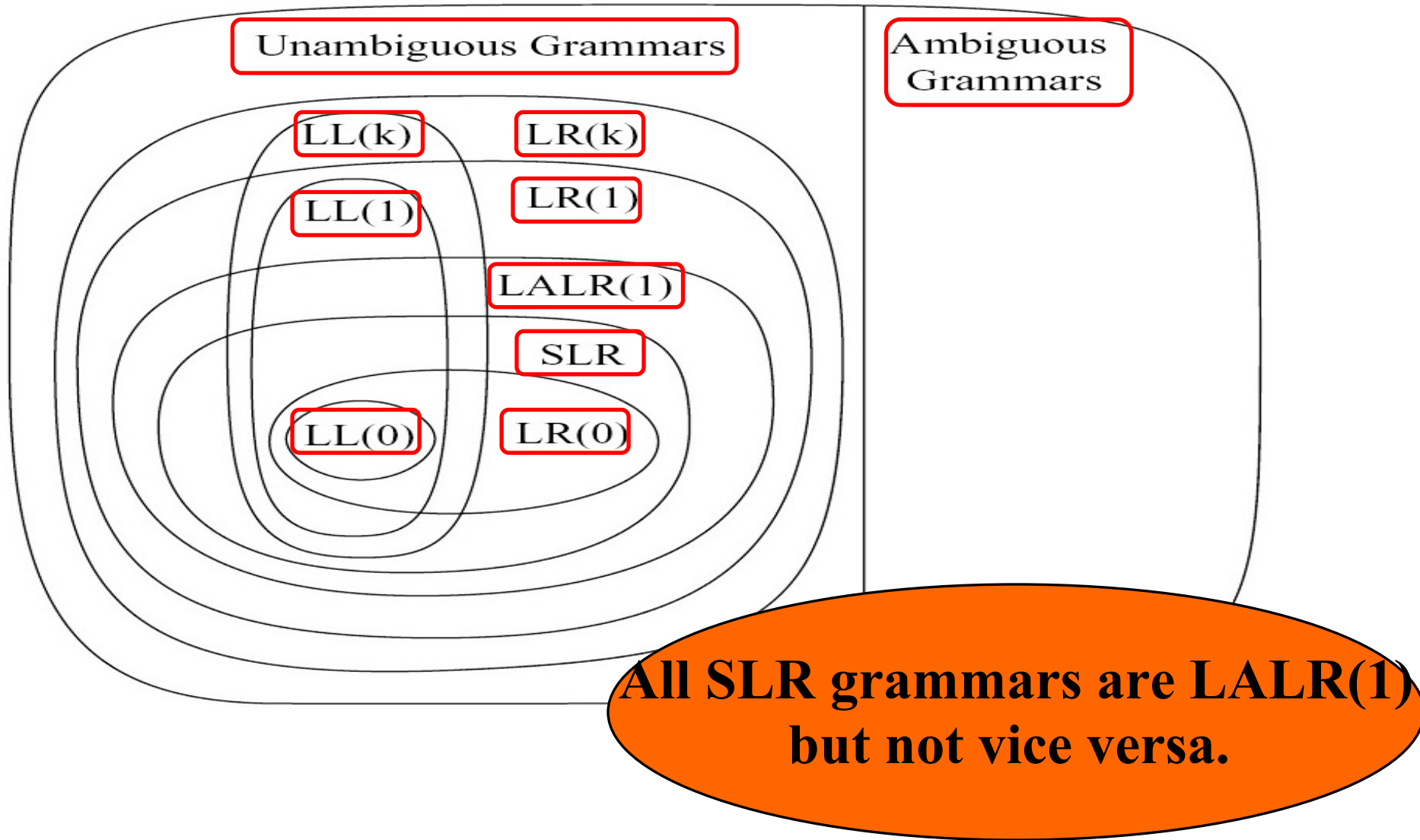
- For some grammars, the **LALR(1) table** contains **reduce-reduce conflicts** where the LR(1) table has none, but in practice the difference matters little.

How about shift-reduce conflicts?

- LALR(1) parsing table requires **less memory** to represent than the LR(1) table

Hierarchy of Grammar Classes

The relationship between several classes of grammars.



LR Parsing of Ambiguous Grammars

- Most programming languages have grammar rules such as

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{other}$

- if a then if b then s1 else s2**

(1) if a then { if b then s1 else s2 }

(2) if a then { if b then s1 } else s2

- In most programming languages, an *else* must match the most recent possible *then*, so interpretation (1) is correct.

$S \rightarrow \text{if } E \text{ then } S .$	else	$\rightarrow$ (2)
$S \rightarrow \text{if } E \text{ then } S . \text{else } S$	(any)	$\rightarrow$ (1)

Shift-Reduce Conflict

LR Parsing of Ambiguous Grammars

One solution:

- The ambiguity can be eliminated by introducing auxiliary non-terminals **M** (for matched if) and **U** (for unmatched if)

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{other}$

$S \rightarrow M$

$S \rightarrow U$

$M \rightarrow \text{if } E \text{ then } M \text{ else } M$

$M \rightarrow \text{other}$

$U \rightarrow \text{if } E \text{ then } S$

$U \rightarrow \text{if } E \text{ then } \boxed{M} \text{ else } U$

M : all then are matched

U : some then is unmatched

This then is unmatched

This then is matched

Why must be M?

LR Parsing of Ambiguous Grammars

Another solution:

- Leave the grammar unchanged. In constructing the parsing table, this conflict should be resolved by shifting (**prefer interpretation (1)**)
- **Caution:** Most shift-reduce conflicts, and probably all reduce-reduce conflicts, are symptoms of an ill-specified grammar, and they should be resolved by eliminating ambiguities.

Exercise

- **3.6, 3.9, 3.13, 3.14**